门诊排队_系统设计说明书

项目名称：智能导诊与挂号导航助手

团队：门诊排队 ｜ 版本：v1.0（2026-11） ｜ 对应工程：`medical-triage-backend`

1 引言

1.1 编写目的

本文档描述「智能导诊与挂号导航助手」的软件系统设计，作为第二次团队作业《系统设计说明书》交付物，

并作为后续编码实现、前后端联调、上线部署与验收的基准。文档采用自顶向下的分层描述：

先给出总体架构，再逐层细化模块设计，最后落到接口、数据与安全约束，

保证设计与代码（`medical-triage-backend`）一一对应、可核查。

1.2 项目背景

项目面向"不知道该挂哪个科、面对挂号平台无从选择"的患者，提供诊前分诊导航的**行动指引**：

患者用自然语言描述症状，系统经过急症拦截、多轮追问与知识检索，输出「主推科室 + 备选科室 +

判断依据 + 挂号指引 + 院内导航 + 沟通话术 + 就医提醒」的结构化报告。

系统**不做在线问诊、不做诊断、不开处方、不承诺疗效**，全程标注"仅供参考，不替代医生诊断"。

需求依据见第一次作业《软件需求规格说明书》与调研数据（问卷 28 份、协和医院实地走访）。

1.3 术语与缩写

缩写/术语	全称/含义
Agentic RAG	以任务流程编排（Agent）组织检索增强生成（RAG）的智能体构建方法
NLU	Natural Language Understanding，自然语言理解（本项目指症状信息抽取）
LLM	Large Language Model，大语言模型（豆包/DeepSeek 等，OpenAI 兼容协议接入）
UUID	Universally Unique Identifier，会话匿名标识
TTL	Time To Live，会话过期时间（默认 3600 秒）
Mock 模式	不调用真实大模型的本地规则模板生成，保证无 Key 可演示全流程
红旗信号	急症高危关键词（依据 WS/T 591—2018《急诊预检分诊》Ⅰ/Ⅱ 级）

科室白名单	`departments` 表中的标准科室名集合，作为 LLM 输出的枚举校验基准
SRS	Software Requirements Specification，软件需求规格说明书（第一次作业交付）

1.4 参考资料

- 2026 秋软工实践团队作业-第二次（原型设计+概要设计）作业要求
- 《软件需求规格说明书（智能导诊与挂号导航助手）》（第一次作业，腾讯文档）
- 《门诊排队_数据库设计说明书》（本作业配套文档）
- 接口说明与联调测试用例表（工程内 `接口说明.md`、`联调测试用例表.md`）
- 《急诊预检分诊》WS/T 591—2018；《医疗卫生机构业务科室分类与代码》CT08.00.00.0002

2 系统总体设计

2.1 设计目标与原则

1. **受控优先**：大模型只做"受约束的生成"，科室枚举、判断依据、输出格式全部由代码约束，防幻觉；

2. **安全底线纯代码**：急症拦截不经过大模型，规则引擎全量单测，宁可多拦不可漏拦；

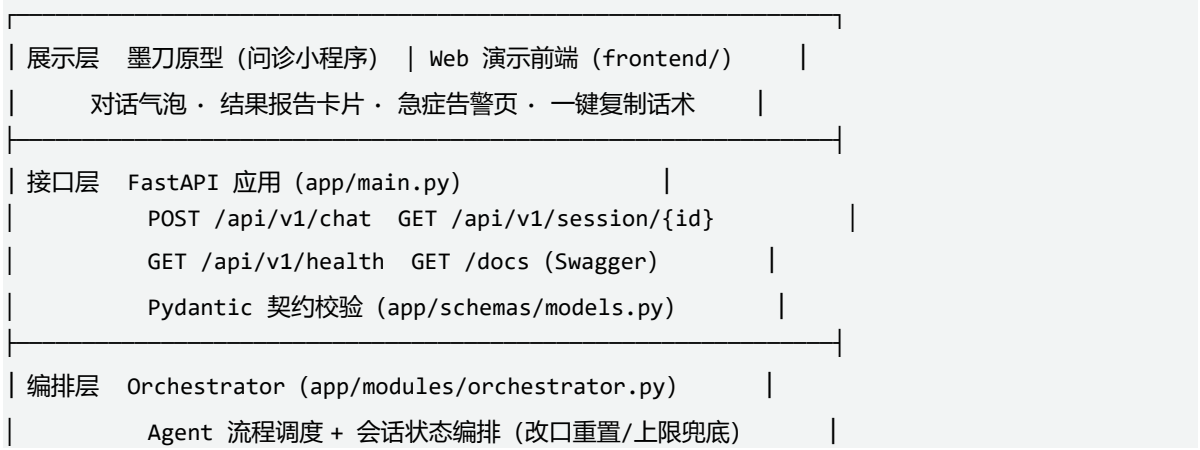
3. **前后端分离**：前端（原型/小程序/Web）与后端仅通过 HTTP JSON 契约交互，科室判断权完全在后端；

4. **知识-代码解耦**：知识库（规则/红旗词/文档）以 JSON 为唯一编辑源，灌入 SQLite 运行，扩充数据不改代码；

5. **可演示可上线**：mock 模式零成本演示；真实大模型/Redis/云服务器通过配置切换，业务代码不动。

2.2 总体架构

系统分为四层，自顶向下：



```
├──────────────────────────────────────────────────────────┤
│ 能力层  ① NLU 信息抽取    app/modules/nlu.py              │
│       ② 急症拦截规则引擎  app/modules/triage.py           │
│       ③ 多轮追问状态机    app/modules/dialog.py           │
│           ④ RAG 知识检索    app/modules/rag.py            │
│           ⑤ LLM 结构化生成   app/core/llm_gateway.py       │
├──────────────────────────────────────────────────────────┤
│ 知识/数据层 SQLite 知识库 (app/knowledge/schema.sql + db/)  │
│           departments/symptom_rules/high_risk_signals/    │
│           hospitals/knowledge_docs · 会话存储 内存/Redis    │
└──────────────────────────────────────────────────────────┘
```

2.3 运行环境与技术选型

组件	方案	说明
语言/框架	Python 3.11+ / FastAPI 0.115	异步 Web 框架，自带 Swagger 文档
数据校验	Pydantic v2.10	请求/响应/生成结果的 Schema 约束
LLM 网关	OpenAI 兼容协议（豆包/DeepSeek/通义等）	可切换 `mock` 本地降级，`httpx` 调用
数据库	SQLite（标准库 sqlite3）	启动自动建表灌种子；上线可平移 PostgreSQL
会话存储	内存字典（默认）/ Redis 5	抽象类 `SessionStore` 统一接口，TTL 1h
知识检索	规则库匹配 + 内置轻量向量（词频-余弦）	纯 Python 实现，可替换为外部向量库
测试	pytest 8.3	44 个用例，含安全词全量回归
部署	本地 uvicorn / Docker	`Dockerfile` 已提供

2.4 设计约束（红线）

- 不提供任何疾病诊断结论、治疗建议与疗效承诺；
- 科室名只能来自 `departments` 白名单枚举，LLM 返回枚举外科室一律拒绝/降级；
- 判断依据只能来自检索资料（规则 reason + 知识文档），不得凭空生成；
- 急症命中强制终止分诊流程，引导线下就医或拨打 120；
- 不收集姓名、手机号、身份证等个人信息，会话以随机 UUID 标识、结束即销毁；
- 全部输出强制携带免责声明"仅供参考，不替代医生诊断"。

3 功能模块设计

3.1 模块划分总览

模块	文件	职责	上层调用方
接口路由	app/api/routes.py	暴露 chat/session/health	展示层

		三个接口	
依赖组装	app/api/deps.py	懒加载单例编排器，注入全部能力模块	路由
契约模型	app/schemas/models.py	Pydantic 请求/响应/结果/会话模型	全局
编排器	app/modules/orchestrator.py	Agent 流程调度、会话状态编排	路由
信息抽取	app/modules/nlu.py	口语 → 结构化症状（双通道）	编排器
急症拦截	app/modules/triage.py	高危词纯代码匹配（安全关键）	编排器
追问状态机	app/modules/dialog.py	缺失字段判定、去重追问、上限兜底	编排器
知识检索	app/modules/rag.py	规则库匹配 + 向量证据检索	编排器
结构化生成	app/core/llm_gateway.py	LLM 生成 + 枚举校验（mock 双轨）	编排器
知识加载	app/knowledge/loader.py	库优先、JSON 兜底的知识读取	各能力模块
数据层	app/db/database.py, seed.py, session_store.py	建库灌种、会话存储	全局

模块依赖单向：展示层 → 接口层 → 编排层 → 能力层 → 知识/数据层，无反向依赖；

能力模块之间不直接互相调用，全部经编排器串联（详见 UML 类图/协作图）。

3.2 接口层设计

- `POST /api/v1/chat`：核心对话接口。请求 `{session_id, message}`；Pydantic 校验

`session_id`/`message` 非空白、`message` 1~4000 字符，非法返回 422 且不创建会话。

- `GET /api/v1/session/{session_id}`：查询会话状态（收集字段/轮数/是否拦截/历史），不存在返回 404。
- `GET /api/v1/health`：健康检查，返回 `status/app/llm_provider/session_store/retriever`。
- 响应统一 `ChatResponse{type, message, data}`，`type ∈ question | result | urgent`（契约详见第 5 章）。
- 跨域：开发期 `allow_origins=["*"]`，上线收紧为部署域名。

3.3 编排层设计（Orchestrator）

一次对话的处理流程（与活动图对应）：

```
用户消息
 ├─ 1）会话状态装载（无则新建 SessionState，history 追加 user 消息）
```

```
├─ 2）急症拦截（纯代码，最先执行）
│   命中 → 状态 interrupted，返回 urgent（含 level/label/keyword）
├─ 3）改口检测（正则："说错了|搞错|弄错|改口|更正|不是…而是"）
│   命中 → 重置 collected/asked/turn，从改口点截取新主诉，历史保留审计
├─ 4）信息抽取（NLU）→ 合并进会话（仅填充空字段，伴随症状去重累加）
├─ 5）追问判定（按 部位→性质→时长→人群 优先级）
│   缺字段且未达上限 → 返回 question（去重不重复问）
├─ 6）检索 + 生成（信息齐全或达上限）
│   全文 = 最近一次改口之后的所有用户消息（跨轮累计匹配）
│   规则检索 → 主推/备选/依据；向量检索 → top5 证据
│     LLM 结构化生成（mock 或真实模型）→ TriageResult
└─ 7）组装响应：question / result / urgent
```

设计要点：**原始文本优先于结构化文本做规则匹配**（"胸口闷"连续词在第一轮出现，第三轮出结果时仍能命中）；

改口后旧主诉不参与检索；**追问上限（默认 3）后无条件兜底出结果**，不无限追问。

3.4 能力层模块设计

3.4.1 NLU 信息抽取（nlu.py）

- **通道一（默认）规则/词典抽取**：部位词典（口语→规范名，长词优先，如"胸口/心口→胸部"）、

性质词典（闷/痛/胀/烧灼/痒…）、时长正则（数字+天/周/月/小时…）、伴随症状词典、人群词典（成人/老人/儿童/孕妇…）。

- **通道二（预留）LLM 兜底**：仅当配置真实大模型时启用，补充规则未抽到的字段；两通道合并去重。
- 输出 `SymptomInfo{part, nature, duration, accompanying[], population}`；

抽不到的字段留空，交由追问状态机处理——"低置信度视为未收集"。

3.4.2 急症拦截（triage.py，安全关键）

- 启动时从 `high_risk_signals` 加载红旗词（417 条：Ⅰ级 emergency 201 / Ⅱ级 urgent 216），

按级别预编译匹配表；

- 匹配顺序先 emergency 后 urgent，多级命中取更紧急者；
- **纯代码实现，不依赖大模型与网络**，命中即返回引导文案（含拨打 120/线下急诊），

全量关键词有自动化回归测试兜底（"宁可多拦不可漏拦"）。

3.4.3 多轮追问状态机（dialog.py）

- 字段收集优先级：部位 → 性质 → 时长 → 人群；伴随症状只主动抽取、不强制追问；
- 每轮至多问 1 个问题，已问过的问题不再重复（`asked` 列表去重）；
- `turn_count` 达上限（`MAX_FOLLOWUP_TURNS`，默认 3）后停止追问，用已收集信息兜底出结果；
- 配合编排层的改口检测：改口时重置状态机，新主诉从头收集。

3.4.4 RAG 知识检索（rag.py）

- **规则库检索（主路径）**：`RuleRetriever` 遍历规则（425 条/1775 个关键词），

用「用户原始文本 + 结构化文本」双候选做关键词包含匹配，命中多条取 priority 最高；

未命中时 mock 生成回退"综合门诊"（兜底科室，仍附免责声明）。

- **向量检索（证据增强）**：`VectorRetriever` 对知识文档做词频向量 + 余弦相似度，

返回 top 5 `RetrievedDoc{text, source, score}` 作为 LLM 生成依据；

纯 Python 实现、零重依赖，接口留出替换外部 Embedding/向量库的位置。

- **科室白名单**：由 `departments` 表导出，注入生成阶段做枚举校验。

3.4.5 LLM 结构化生成（llm_gateway.py）

- **openai 模式**：OpenAI 兼容 `/chat/completions` 调用（豆包/DeepSeek 等），

`temperature=0.2`、`response_format=json_object`；系统提示词硬性约束：

①科室只从给定枚举选；②依据只来自检索资料；③不提供治疗建议；④按 8 个字段输出 JSON。

- **代码侧校验（防幻觉关键）**：解析 JSON（容错：抽取第一个 `{...}` 块；

数组字段若被模型返回成字符串则自动归一为单元素数组）；`recommended_department`

不在白名单 → 抛错拒绝，绝不静默放行编造科室。

- **mock 模式**：本地规则模板按 `rule_hit` 组装完整 TriageResult（科室/依据/挂号指引/导航/话术/提醒），

无 Key 无网络也能演示全流程；切换真实模型只改 `.env` 四个配置，业务代码不动。

- 输出 `TriageResult` 经 Pydantic 校验，`disclaimer` 默认"仅供参考，不替代医生诊断"。

3.5 数据层设计

见《门诊排队_数据库设计说明书》。要点：5 张知识表（SQLite，启动自动建表灌种子）；

会话状态与任务结果不落库（内存/Redis，TTL 1h）；JSON 为知识唯一编辑源、

`reseed.py`/`import_*.py` 提供重灌与增量导入。

4 领域 Agent 设计

4.1 领域知识及来源

知识	内容	来源（source 字段逐条标注）
科室白名单	32 个标准科室名（24 主推 + 8 备选）	CT08.00.00.0002 标准科室目录（code 待补）
症状-科室规则	425 条规则 / 1775 个症状关键词	人工整理 392；CMeKG/中文主诉-专科数据集第二批扩充 1005；公开疾病图谱统计关联 378（priority=40，保留支持数/占比/关联疾病供审核）
急症红旗信号	417 条（Ⅰ级 201 / Ⅱ级 216）	WS/T 591—2018《急诊预检分诊》
医院挂号渠道	福州 3 家三甲（协和/附一/省立）	医院官网/公众号/114 平台
检索证据文档	20 条导诊知识切片	公开导诊资料

4.2 Agent 任务流程（Agentic RAG）

智能体完成的任务是**「把模糊的口语主诉变成一份可执行的行动指引」**，流程与模块一一对应：

1. **理解**（NLU）：口语 → 结构化症状字段，规则可审计、LLM 可兜底；

2. **守护**（急症拦截）：红旗词纯代码匹配，先于一切流程，命中即终止并引导 120；

3. **追问**（状态机）：按优先级补齐缺失字段，去重、限轮、支持改口重置；

4. **检索**（RAG）：规则库给出主推/备选科室与依据，向量检索提供证据语料；

5. **生成**（受控 LLM）：提示词 + 白名单 + Schema 三重约束输出结构化报告；

6. **交付**（结构化响应）：前端直接渲染报告卡片/急症告警，附免责声明。

4.3 工具调用与流程编排

本项目把"模块即工具"：NLU、拦截、追问、检索、生成五个能力模块相当于 Agent 的工具箱，

由 Orchestrator 按会话状态决定调用次序与分支（急症短路、改口重置、上限兜底），

即"流程编排"式 Agentic RAG。相比让大模型自由决定调用哪个工具，编排权在代码手里，

行为可预测、可测试、可审计——这对医疗场景的容错要求是必须的。

4.4 智能体与软件功能的集成关系

Agent 环节	用户可感知的软件功能	数据/接口衔接
信息抽取 + 追问	对话式输入与智能追问气泡	`POST /chat` → `question` 响应
急症拦截	急症红色告警页、拨打 120 按钮	`type=urgent` + level/label
知识检索 + 结构化生成	报告卡片：科室/依据/指引/话术/提醒	`type=result` + `data` 八字段
会话状态机	断点恢复（`GET /session`）、新会话	内存/Redis 会话
免责与隐私	页面固定免责声明、UUID 匿名会话	`disclaimer` 字段 + session_id

4.5 定制前后能力对比（典型案例）

以"我肚子疼了两天，偶尔反酸"为例：

对比项	通用大模型直问	本系统（领域定制后）
科室依据	可能凭记忆猜测、甚至编造科室	只能从 1775 条规则命中的科室（消化内科）选择，附判断依据
追问	不追问或追问随机	按部位→性质→时长→人群确定性追问，不重复、有上限
急症	可能继续聊天式安抚	"剧烈胸痛伴出冷汗"等 417 个红旗词立即强制引导 120
输出	长段落文字，需自行提炼	8 字段结构化报告，前端直接渲染卡片 + 一键复制话术
幻觉防线	无	白名单枚举校验 + 依据只来自检索 + Schema 校验

5 接口设计

5.1 接口总览

方法	路径	说明	主要错误
POST	/api/v1/chat	发起/继续导诊对话	422（空/空白/超 4000 字/缺字段）
GET	/api/v1/session/{session_id}	查询会话状态	404（会话不存在）
GET	/api/v1/health	健康检查	—
GET	/docs	Swagger 交互文档（开发期）	—

5.2 POST /api/v1/chat

请求：

```
{ "session_id": "前端生成的 UUID", "message": "我胸口闷，喘不上气" }
```

响应三态：

- `type=question`：`data.collected` 为已收集字段，`message` 为追问文案；
- `type=result`：`data` 含 `recommended_department / alternative_departments / basis / booking_guide / navigation / dialogue_script / reminders / disclaimer / evidence`；
- `type=urgent`：`data` 含 `level（emergency|urgent）/ label / keyword`，`message` 为急症引导（含 120）。

完整契约与前端对接示例见工程 `接口说明.md`。

5.3 会话与健康检查

- `GET /api/v1/session/{id}`：返回 `session_id/status/collected/turn_count/interrupted/history`，供前端刷新恢复；
- `GET /api/v1/health`：返回 `status=ok/app/llm_provider/session_store/retriever`，供前端检测后端在线状态。

6 数据库设计（摘要）

详见《门诊排队_数据库设计说明书》。要点：5 张表 + 3 索引；`departments.name` 作为 `symptom_rules.recommended_dept` 外键基准形成防幻觉闭环；`alt_depts` JSON 反范式（有意为之）；

会话与任务结果不落库（内存/Redis，TTL 1h）；JSON 唯一编辑源 + 幂等种子灌入。

7 界面设计

- **墨刀原型（用户端问诊小程序）**：`https://modao.cc/proto/djKmUlT6to17c228Y8MJ7d/sharing?view_mode=read_only&screen=rbpVa6d0lkNEKZqxS`

页面：启动页 → 对话页（追问/报告卡片/复制话术）→ 急症告警页（拨打 120/返回修改）。

- **Web 演示前端**（工程 `frontend/index.html`，根路径访问）：真实调用后端，含结果报告卡片（主推+备选+依据+挂号指引+院内导航+话术复制+提醒+免责声明）、急症红色告警、新会话、加载态与输入校验提示；与原型同一套视觉语义（绿色医疗主题）。
- 交互规则：输入非空校验、发送中禁用按钮防重复提交、`disclaimer` 固定展示在结果底部、急症告警后终止常规对话（需新会话重新开始）。

8 安全与合规设计

1. **急症拦截纯代码**：不依赖大模型，417 个红旗词全量自动化回归（"宁可多拦不可漏拦"）；

2. **科室枚举校验**：LLM 只能从白名单选科室，返回枚举外科室直接拒绝，杜绝"一本正经编科室"；

3. **检索约束生成**：判断依据只来自规则与知识文档，提示词明令禁止诊断结论与治疗建议；

4. **结构化输出校验**：`TriageResult` 走 Pydantic Schema，非法输出拒绝/降级；

5. **隐私最小化**：不收集姓名/手机号/身份证；会话以随机 UUID 标识；会话结束数据即刻销毁（TTL 1h）；

6. **合规文案**：全链路强制标注"仅供参考，不替代医生诊断"；急症判断只引导线下就医/120，

线上不给出任何实质性医疗承诺；

7. **数据来源可追溯**：知识库每条数据 `source` 字段标注出处，图谱关联数据保留支持数与占比供审核。

9 测试设计

层级	内容	现状
单元测试	信息抽取（部位归一化/性质/时长/人群）、规则命中、追问状态机、拦截器	通过
回归测试	原始文本跨轮匹配、成年人识别、红旗新词全量命中	通过
边界测试	空/空白/超长/标点/emoji 输入、改口修正、追问上限兜底、会话隔离	通过
API 契约测试	422 边界（空、4001 字、缺字段）、404 会话、急症中途拦截	通过
联调用例	`联调测试用例表.md` 七类 40+ 用例（正常流程/急症/追问/边界/LLM 校验/接口）	自动化 44/44 通过；浏览器端与真实大模型联调待执行

通过标准（联调用例表第 7 节）：Ⅰ级急症用例 100% 命中且文案含"120"；追问上限不无限问；

改口结果跟随最新主诉；LLM 枚举校验不静默放行；异常输入无 500。执行方式：

`python -m pytest tests -q`（工程 `run_tests.ps1`）。

10 部署设计

- **本地演示**：`run.ps1` 一键完成 venv 创建、依赖安装与 `uvicorn app.main:app --host 0.0.0.0 --port 8000` 启动；

默认 `LLM_PROVIDER=mock` 无 Key 可跑全流程；演示页 `/`，接口文档 `/docs`。

- **容器部署**：`Dockerfile` 已提供（`docker build` / `docker run -p 8000:8000`）。
- **上线规划**：真实大模型（豆包/DeepSeek）改 `.env` 四配置；`SESSION_STORE=redis` 支持多实例；

云服务器 + 域名/HTTPS；SQLite → PostgreSQL 平移（schema.sql 为标准语法）。

11 系统设计检查清单（对照作业评分点）

- [x] 分层自顶向下描述，层越深细节越具体（第 2→3 章）
- [x] 用例图/活动图/类图/时序图/协作图（见《智能导诊系统-UML 图稿》，其中类图/时序图/协作图/活动图

以本说明书第 3 章模块设计为准最终更新）

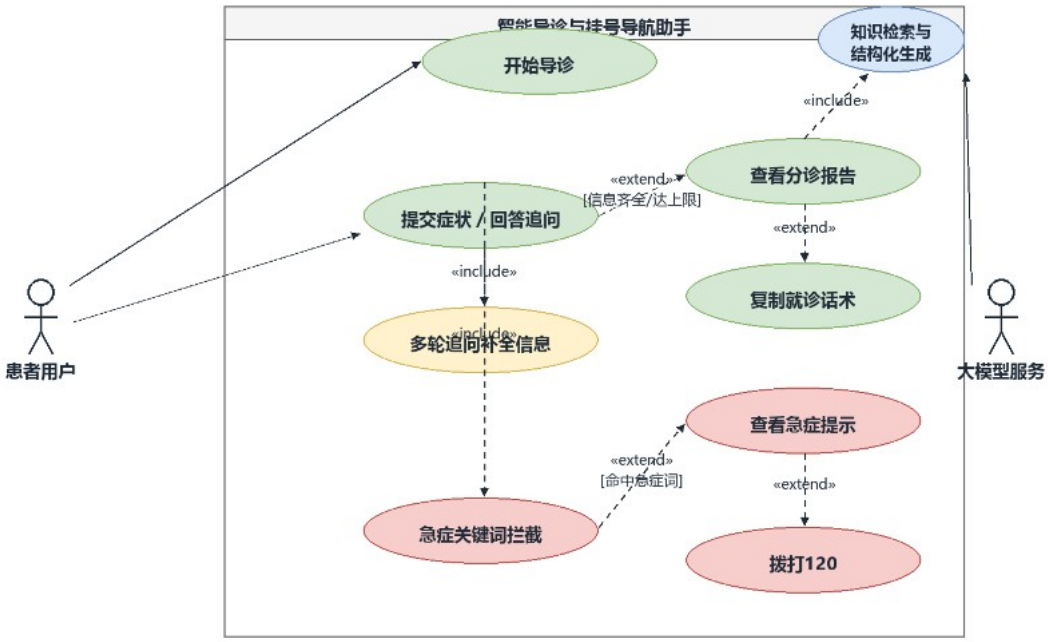
- [x] 领域 Agent 与软件系统的集成方式（第 4 章：知识来源/任务流程/编排/功能映射/定制前后对比）
- [x] 数据库 ER 图、关系模型、ORM 映射（《数据库设计说明书》第 1~2 章）
- [x] 领域知识、用户交互记录、任务执行结果的存储与管理（《数据库设计说明书》第 3 章）
- [x] 接口契约与前端对接（第 5 章 + `接口说明.md`）
- [x] 一致缩写与术语定义（1.3 节）

文档版本说明：本文档依据整合后的可运行工程 `medical-triage-backend` 编写，

与代码事实一致（44 测试全绿、接口四链路实测通过）。

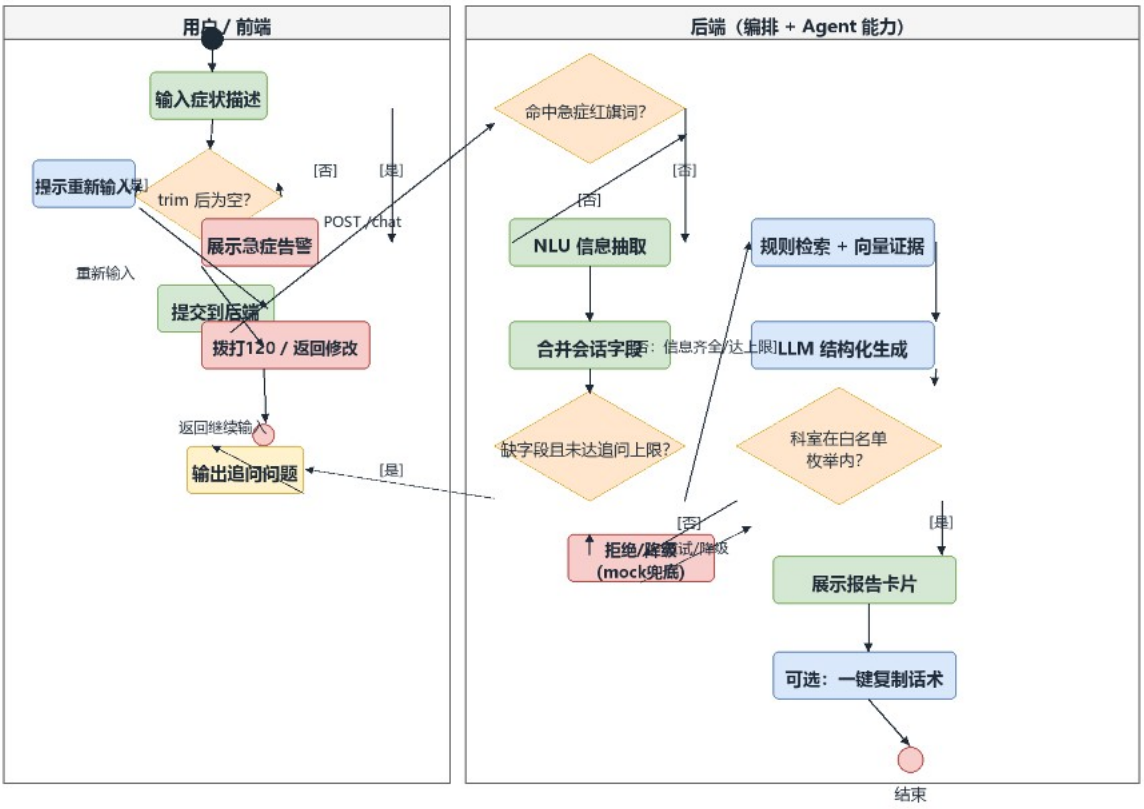
附录 A　UML 设计图（与 drawio 源文件同源）

图 A-1 用例图



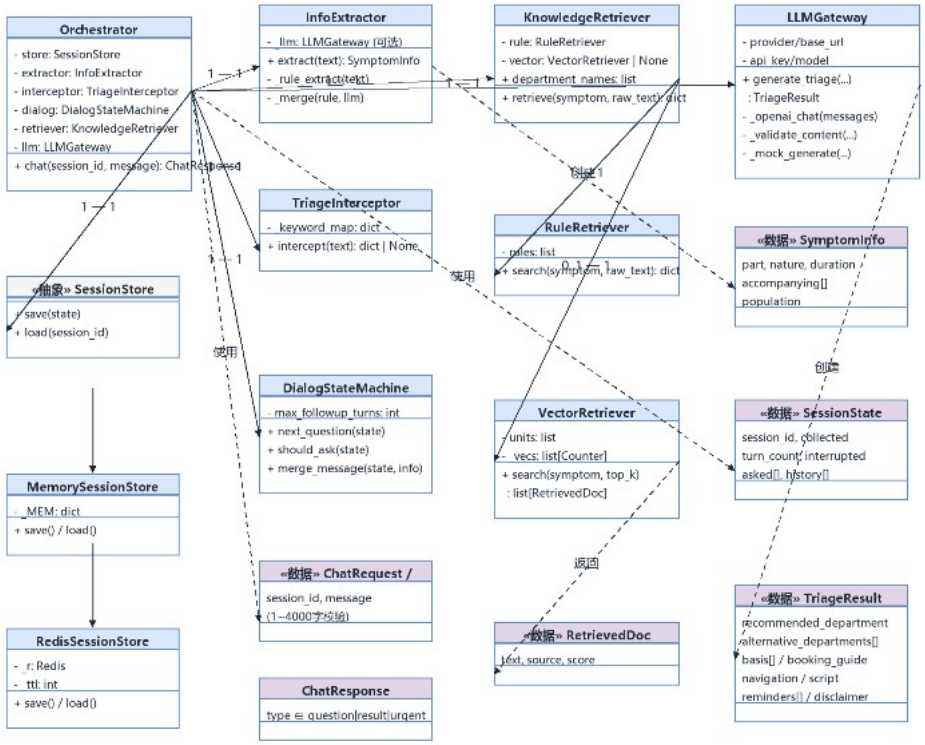
参与者：患者用户、大模型服务（外部）； 9 个用例及 include/extend 关系。

图 A-2 活动图



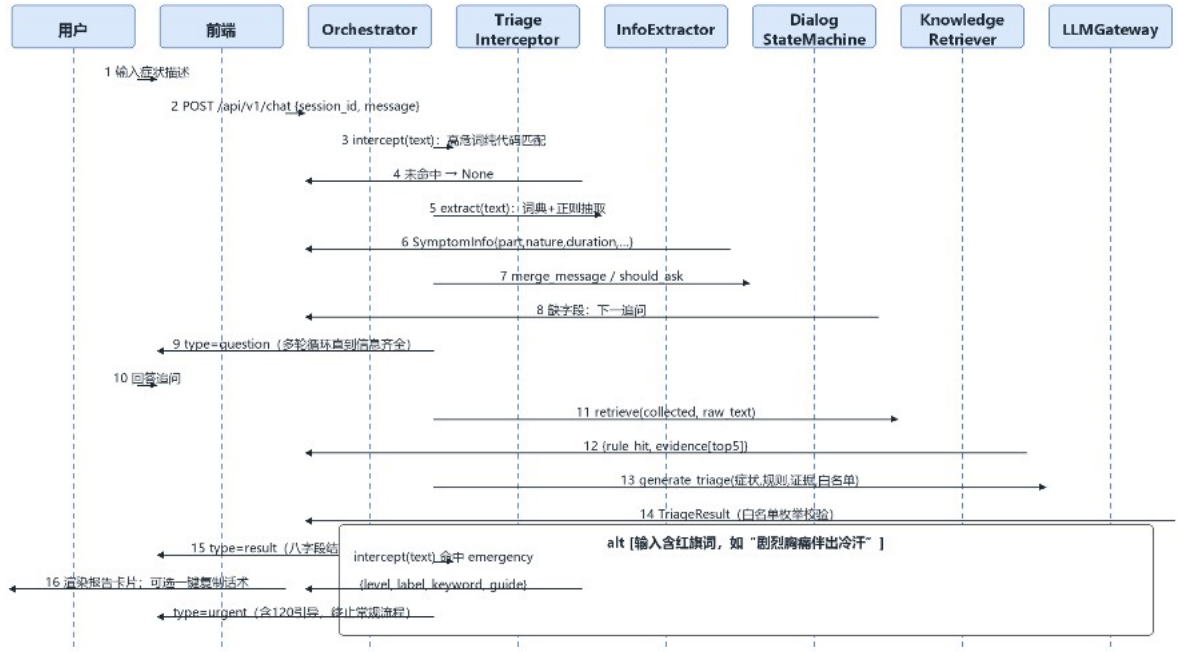
用户/前端与后端双泳道：输入校验、急症短路、多轮追问循环、白名单校验与降级分支。

图 A-3 类图



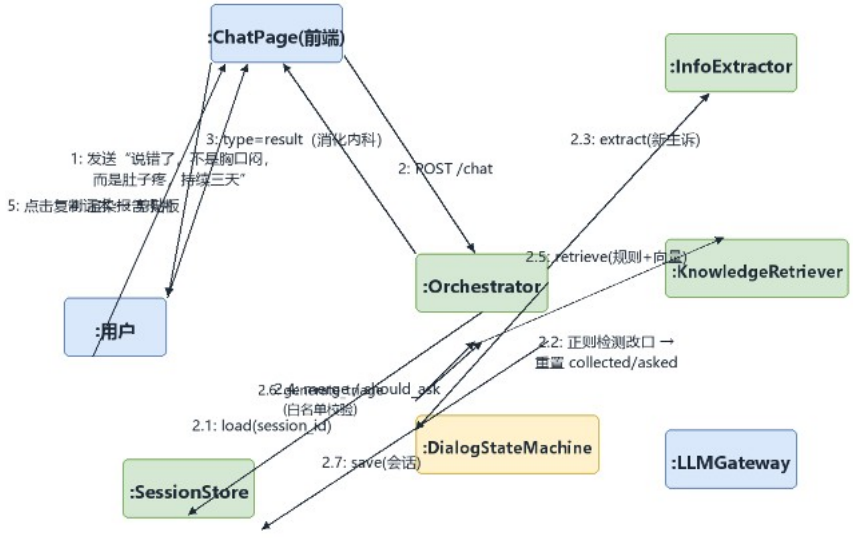
编排器与五个能力模块类、会话存储类及六个 Pydantic 数据类的结构与关联。

图 A-4 时序图



Agentic RAG 完整时序（抽取→追问→检索→生成→校验），alt 片段为急症拦截路径。

图 A-5 协作图



"改口修正"场景下七个对象的协作与编号消息流。